PlanCompiler: A Deterministic Compilation Architecture for Structured Multi-Step LLM Pipelines

Pranav Harikumar
github.com/prnvh/plancompiler

Abstract

Large language models (LLMs) are brittle in multi-step structured workflows, where errors compound across sequential transformations, validation steps, and stateful operations such as SQL persistence. We present PlanCompiler, a deterministic compilation architecture for structured LLM pipelines that separates planning from execution using typed node registries, static graph validation, and topological compilation. Rather than relying on autoregressive chaining at runtime, the system executes a prevalidated workflow graph with explicit parameter and schema constraints.

We evaluate the approach on 300 tasks across six benchmark sets covering increasing workflow depth, SQL roundtrip persistence, and schema-drift stress tests. On depth-stratified benchmarks, the compiled system achieves 100% accuracy on Sets A and B, 88% on Set C, and 96% on Set D, outperforming direct code-generation baselines from GPT-4.1 and Claude Sonnet. On Schema trap tasks, it achieves 44/50 compared with 20/50 for GPT-4.1 and 26/50 for Claude. Across the full suite, the compiler consumes approximately \$0.356 in total inference cost, versus \$2.140 for GPT-4.1 and \$18.391 for Claude, while maintaining competitive latency and avoiding the timeout-heavy failure modes observed in free-form baselines.

These results indicate that, in this benchmark setting, deterministic compilation improves first-pass reliability and cost efficiency for structured multi-step LLM pipelines, while concentrating remaining failures at the planner–semantics boundary rather than in execution-time structural errors.

Keywords: LLM, program synthesis, workflow compilation, reliability, determinism, structured generation

1 Introduction

Large language models have demonstrated substantial capability in generating executable code from natural language descriptions. In single step settings such as producing a sorting function, writing a SQL query, implementing a utility, this capability is well-established and increasingly reliable. The more difficult problem arises when tasks require multiple sequential operations: ingesting structured data, applying a series of transformations, persisting results to a database, and exporting a final artifact. In these multi-step settings, free-form code generation exhibits a characteristic failure mode: errors accumulate across steps. A hallucinated import in step two

propagates through steps three and four. A mismatched column name introduced at ingestion causes silent failures at query time. Naming conventions drift between the function a model generates and the function it calls. Each individual step may look locally correct while the composed pipeline fails.

Existing approaches address this in one of two ways. Tool-orchestration frameworks such as LLMCompiler treat the model's output as a DAG of function calls and focus on scheduling and parallel execution, improving latency and cost while accepting execution failure as a runtime concern. NL-to-pipeline synthesis systems such as Text-to-Pipeline define a domain-specific language for data preparation and compile model output to Pandas or SQL, demonstrating that structured intermediate representations improve over direct code generation. Both lines of work normalize the pattern of generating a symbolic representation and executing it deterministically. What is less explicit in this literature is the question of what guarantee the representation boundary provides: which failure modes it eliminates by construction, which it defers to runtime, and which it cannot catch at all.

We describe PlanCompiler, a deterministic compilation architecture that treats this question as its central design concern. The system imposes a strict separation between planning and execution. The LLM is permitted to do exactly one thing: select nodes from a pre-verified registry and supply their required parameters, expressed as a typed JSON plan. The registry fixes the set of callable primitives; the LLM cannot invent new ones. A deterministic validator then runs seven structural checks on the plan—node existence, edge validity, type compatibility, acyclicity, orphan detection, input arity, and required parameter presence—before any execution occurs. If validation passes, a deterministic compiler assembles an executable Python program from pre-verified node templates using topological sort. The LLM is not called again after the plan is emitted. There is no repair loop, no output inspection, no autoregressive chaining at runtime. First-pass success under this architecture is the central empirical claim.

We evaluate the system on six benchmark sets totaling 300 tasks, each designed to stress a different aspect of multi-step workflow execution. Sets A–D vary pipeline depth from short to long compositions, while the remaining sets test robustness under schema perturbations and SQL-mediated state transitions. Across these benchmarks, the compiled system achieves high first-pass success rates and remains substantially more reliable than the free-form code generation baselines.

A secondary finding emerges from the evaluation that appears to reflect a broader design issue rather than an isolated benchmark artifact. The node registry constrains all surfaces of the plan except one: the raw SQL string passed to the QueryEngine node. In complex tasks where the Aggregator node would be the structurally correct choice, the planner systematically routes computation into the SQL parameter instead—exploiting the only unconstrained surface available. We document this as *constraint evasion under partial enforcement*: in our setting, when most surfaces are constrained and one expressive surface remains open, the planner systematically routes work through that surface. We view this as a design lesson that may apply more broadly to partially constrained generation systems.

1.1 Contributions

The contributions of this paper are three. First, we specify a typed intermediate representation and compiler architecture for LLM program synthesis that eliminates hallucinated imports and many forms of naming and wiring drift by construction rather than by prompting. Second, we present a reliability-focused evaluation protocol with explicit success criteria and a structured failure taxonomy, showing that the architecture maintains high first-pass accuracy across the benchmark sets while free-form generation remains substantially less reliable. Third, we identify and characterize constraint evasion in this partially constrained architecture, show that it recurs across multiple benchmark settings, and argue that it should be treated as a design-level concern rather than only as a prompt-engineering issue.

2 Related Work

This work sits at the intersection of agent orchestration frameworks, natural-language-to-pipeline synthesis, and correctness enforcement for LLM-generated execution. We position PlanCompiler relative to each below.

LangChain and LlamaIndex provide widely used abstractions for building LLM-powered workflows over tools and external components [1, 2]. These frameworks make multi-step composition easier, but they do not impose a hard static correctness boundary before execution. Tool selection and sequencing remain primarily model-driven, and the system can still generate or route arbitrary orchestration logic between calls. In that setting, failures are typically handled at runtime rather than ruled out structurally before execution begins.

Hugging Face's smolagents takes this openness further with an explicitly code-first design [3]. Its `CodeAgent` emits executable Python, allowing loops, conditionals, and unrestricted logic between tool calls. While tools themselves may be typed or structured, the orchestration layer remains free-form generated code. PlanCompiler differs along a single architectural axis: the model is not allowed to write execution logic at all. It may only select nodes from a closed registry and supply required parameters. Execution order, node wiring, and runnable code are all derived deterministically after the plan is accepted.

LLMCompiler is structurally close to our system in that it treats tool use as a compiler problem and has the model produce a DAG of function calls for later execution [4]. The similarity, however, is mainly architectural. LLMCompiler is primarily motivated by efficiency, especially parallel scheduling to reduce latency and cost. In PlanCompiler, planner–executor separation serves a different purpose: it creates a structural correctness boundary. We are not mainly trying to speed up execution; we are trying to ensure that planning and execution are separated enough for structural guarantees to be enforced before any code runs.

OrchDAG is also closely related [5]. It studies complex tool orchestration through DAG-structured plans and includes rule-based verification mechanisms such as structural and dependency checks. That is adjacent to our setting, since both systems rely on structured plans whose internal references can be validated. The difference is where verification sits in the overall

design. In OrchDAG, verification primarily supports data generation, evaluation, and training. In PlanCompiler, validation is a runtime enforcement boundary: no plan is compiled unless it passes all seven checks, and that boundary is central to our reliability claims.

Text-to-Pipeline is the closest prior line of work at the architectural level [6]. Both approaches generate a structured intermediate representation and execute it deterministically rather than emitting unrestricted code directly. The key difference lies in the form of the intermediate representation and in the degree of model freedom. Text-to-Pipeline allows the model to generate DSL programs with substantial expressive flexibility. In PlanCompiler, the model solves a constrained selection problem over a closed node registry and cannot invent new primitives. Text-to-Pipeline evaluates functional equivalence between generated and reference pipelines, whereas our evaluation focuses on first-pass task success under increasing structural complexity, with explicit success criteria and a documented failure taxonomy.

AutoPandas also addresses synthesis over a fixed computational surface, using guided search and learned heuristics to construct programs over the pandas API [7, 8]. The relationship to our work is clearest at the problem-framing level: both systems generate structured programs over a predefined set of operations. The methods are different, however. AutoPandas is a search-based synthesis system over a broad API surface, whereas PlanCompiler is an LLM-based planner over a much smaller and more tightly constrained registry.

Jigsaw improves generated data-manipulation code through structured post-processing and repair [9]. Its central lesson is that analysis after generation can recover correctness from imperfect outputs. Our approach moves the correction boundary earlier. Rather than repairing invalid outputs after code has already been produced, we reject structurally invalid plans before compilation occurs. The compiler therefore never receives an invalid plan.

Recent work on governed agent execution, including POLARIS, treats agent behavior as typed planning under explicit policy constraints, with validator-guarded execution and strong auditability [10]. This broader direction helps normalize the combination of structured planning, explicit validation, and governed execution in practical deployment settings.

Our contribution relative to this literature is to turn these design patterns into an explicit execution architecture with enforceable structural guarantees. PlanCompiler places a hard validation boundary between planning and execution: the model may propose a plan, but no code is generated or run unless that plan satisfies a closed set of structural checks. The empirical evaluation is intended to test the architecture's boundary conditions, showing which failure modes disappear under this boundary and which persist. In particular, the QueryEngine evasion result—where partial enforcement causes the model to route work into the last unconstrained surface—identifies a practically important weakness that may arise in partially constrained generation systems.

3 Method

3.1 System Architecture

PlanCompiler consists of five components arranged in a strictly ordered pipeline. Only one component involves a language model call. All subsequent system logic is non-generative; validation and compilation are deterministic, and execution is deterministic given the emitted program, fixtures, and environment.

The planner receives a natural-language task description together with the typed node registry and emits a JSON plan specifying node selections, parameter bindings, and edges. This plan is then passed to a static validator that checks structural correctness before any code is generated. Only validated plans are compiled into executable Python assembled from pre-verified node templates.

3.2 Graph Specification

A node is a typed computational primitive defined as a Pydantic model with seven fields: name, description, input_type, output_type, template_path, required_params, and function_name. Every field is fixed at registry definition time and left untouched at runtime. The function_name field is stored explicitly — the compiler uses it directly, with no string manipulation, snake_case conversion, or regex derivation from the node name. This is a deliberate design choice: the mapping from node to callable is a registry fact, not a naming convention. A separate template_path field points to a pre-written, pre-verified Python file on disk. The compiler reads this file verbatim; the LLM never sees template contents.

An edge is a directed pair [source, target] asserting that the output of source is passed as the first positional argument to target. Edges are the only mechanism for data flow between nodes — there is no shared mutable in-memory state, no global context, no implicit passing. The plan schema uses node names as unique identifiers, meaning a given node type can appear at most once per plan. Repeated use of the same primitive — for example, two sort stages or two instances of the same database node type — is structurally inexpressible in the current schema; this is a documented architectural constraint.

Parameters are key-value pairs attached to a node in the plan's parameters dict. They become keyword arguments in the compiled execution call. Parameters are LLM-supplied; the registry declares which parameters are required, and the validator enforces their presence. The compiler emits parameter values using repr(), so planner-supplied values are inserted directly into the emitted code as Python literals rather than being regenerated or transformed by a second model step. There are no optional parameters with defaults in the current registry, by design. The DataSorter case during development showed that optional parameters with defaults cause silent wrong-output failures: a task specifying descending sort would pass file_has_column criteria while producing incorrectly ordered output. Making ascending a required parameter forces the validator to catch omissions rather than letting them silently produce incorrect output.

The type system defines five types: FilePath, DataFrame, DBHandle, HTTPResponse, and ANY, implemented as a string enum so they serialize cleanly in JSON and error messages. An edge is type-compatible if source.output_type == target.input_type, or if either side is ANY. The ANY wildcard exists specifically for Logger (a passthrough node) and ErrorHandler (which accepts any input). Type compatibility is checked statically before any execution begins. A type mismatch is a hard abort, not a warning.

3.3 Planner

The planner is the only component that involves a language model call. It receives a natural language task description alongside a serialized representation of the node registry — names, descriptions, input/output types, and required parameters — and returns a normalized JSON plan. The plan schema specifies four fields: nodes (an ordered list of registry node names), edges (directed pairs defining data flow), parameters (key-value pairs per node).

The planner includes a normalization step that handles non-standard LLM output formats before the plan reaches the validator: nodes returned as dicts with type and params fields, edges returned as dicts with from and to fields, and node names in snake_case rather than CamelCase are all normalized to the canonical schema. This normalization is applied before validation, not after — the validator always receives a structurally normalized plan.

3.4 Validation Layer

The validator is the primary explicit rejection boundary for plans. If a plan passes all seven checks, it is guaranteed to satisfy the compiler's structural preconditions. There is no partial validation, no warnings-only mode, and no retry. Seven checks run in strict order, with early abort on CHECK 1 because subsequent checks assume node existence.

CHECK 1 (node existence) binds the plan to the registry: any node name not present in NODE_REGISTRY is rejected immediately. CHECK 2 (edge validity) confirms that every node referenced in an edge is declared in the plan's node list. CHECK 3 (type compatibility) compares output and input types across every edge, with the ANY wildcard exempting either direction. CHECK 4 (acyclicity) runs a topological sort via Kahn's algorithm; if not all nodes are visited, a cycle exists and the plan is rejected. CHECK 5 (no orphans) ensures every node in a multi-node plan appears in at least one edge. CHECK 6 (input arity) enforces that every non-entry-point node has exactly one inbound edge, ruling out fan-in structures the current execution model does not support. CHECK 7 (required parameters) verifies the presence of every required parameter for every node, checking presence rather than validity — the validator confirms that condition is present for DataFilter but does not evaluate whether the expression is a valid pandas query string.

The validator exposes topological_sort() as a public function used directly by the compiler. This is the only shared interface between the two components.

3.5 Compilation

The compilation stage in PlanCompiler is called only after the validator returns clean. It reruns validate_plan internally as a defensive check — validation occurs twice, and the compiler does not trust that callers have pre-validated. Topological sort via Kahn's algorithm produces a deterministic linear execution order from the DAG. For the linear pipelines that constitute all current benchmark tasks, this is simply the node list in dependency order.

Template assembly proceeds as follows: for each node in topological order, the compiler reads the corresponding template file from disk verbatim and appends it to the output buffer. There is no string manipulation of template content and no interpolation. The execution block is auto-generated by default: the compiler iterates the topological order, looks up each node's function_name from the registry directly, identifies the predecessor via the edges map, and emits code of the form out_<function_name> = <function_name>(out_<predecessor>, **params). The last node's output is printed.

3.6 Execution Model

Execution is a single Python subprocess invocation of the compiled app.py. There is no intermediate interpreter, virtual machine, or runtime planning layer between the emitted program and execution. Inter-node execution state passes through function return values: each node function takes the previous node's output as its first positional argument and returns a new value. There is no shared mutable in-memory state between nodes and no implicit side-channel communication. Once app.py is emitted, the LLM is not called again. There is no output inspection loop, no repair prompt, and no autoregressive chaining at runtime. If execution fails, it fails. The reliability claim is about first-pass success, not success after correction. Adding a repair loop would likely reduce non-runtime failures, but it would change the claim being evaluated: the present results measure first-pass execution success, not success after iterative correction.

Determinism is a property of the compilation stage rather than the full end-to-end system: given the same valid plan and the same input fixtures, the compiler always emits identical `app.py` content, verifiable by output hash. The remaining source of non-determinism is the planner itself, which may produce different valid plans across runs. By contrast, free-form baseline generation showed small but measurable multi-run variance even at temperature=0; we analyze this in Section 5.

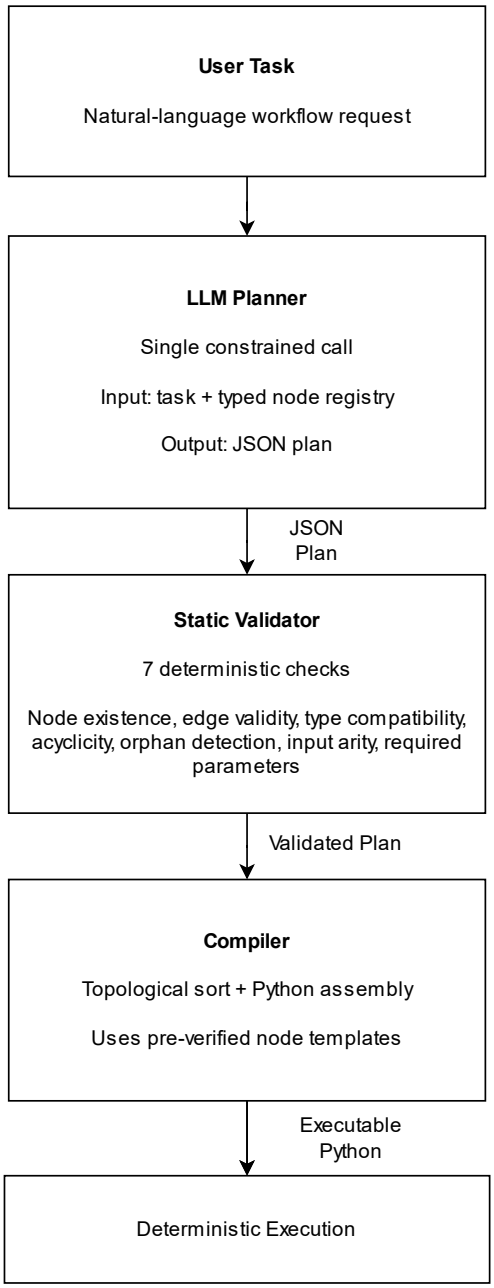

Figure 1: Overview of PlanCompiler. A single LLM call produces a structured JSON plan over a typed node registry. The plan is validated using deterministic structural checks before compilation. Only validated plans are compiled into executable Python and run.

4 Experimental Setup

4.1 Benchmark Design

The benchmark measures first-pass success rate: the fraction of tasks where the full pipeline — plan, validate, compile, execute, criteria check — succeeds end-to-end with zero human intervention. A task is scored as a success only if every criterion passes on the first attempt. There is no repair loop, no retry, and no partial credit.

For each task, the harness tracks: (1) **success/failure** against all criteria, (2) **planner latency and token usage** (the planner uses GPT-4o-mini at temperature=0, measuring input/output tokens and USD cost), (3) **end-to-end wall-clock latency** from plan to execution completion, and (4) **compilation success** (whether generated code is executable). Per-task measurements enable detailed failure analysis across pipeline stages.

Tasks are organized into six sets totaling 300 tasks. Sets A through D approximate a complexity ladder stratified primarily by pipeline length. Sets E and F are specialized stress tests targeting distinct failure modes.

Set	Depth	Count	Focus
A	3–5 nodes	50	Shallow pipelines. Ingestion, single transforms, export.
B	5–8 nodes	50	Medium pipelines. Chained transforms, SQLite roundtrips, aggregations.
C	8–10 nodes	50	Long pipelines. Multi-step SQLite, post-query transforms, complex aggregations.
D	10+ nodes	50	Maximum complexity within library scope.
E	Mixed	50	Schema trap tasks. Null handling, type casting, schema validation chains.
F	Mixed	50	SQL roundtrip tasks. CSV/JSON → SQLite → export data integrity.

Each set contains 30 original tasks and 20 probe tasks designed to stress aggregation routing, Logger type propagation, and SQLite persistence. Probe tasks use the same evaluation protocol and success criteria as original tasks.

All planning operations run at temperature=0 to reduce sampling variance in planner output. Token counts and latency measure pure planning cost without retry or correction loops.

4.2 Success Criteria

The benchmark harness supports five criterion types, evaluated after each task execution. All criteria for a task must pass; if any criterion fails, the task is marked as failed.

Criterion	Check
file_exists	Output file exists at the specified relative path (from run_dir). Path matching is exact and case-sensitive.
file_has_column	Tabular output (CSV or JSON) contains a column with the exact name specified. Column name matching is case-sensitive.
file_row_count	Tabular output contains exactly the specified number of rows (determined by loading the file and calling len(df)).
stdout_contains	Process stdout contains the specified substring (exact match, case-sensitive, no regex).
file_column_sorted	Named column is monotonically sorted in the specified direction (ascending or descending). NaN values are excluded from the sort check; ties are allowed (i.e., non-strict monotonic ordering).

file_column_sorted was added to prevent sort-direction correctness bugs. Prior to this, tasks specifying sort direction were under-verified: a pipeline that sorted ascending when descending was required would pass file_has_column criteria while producing incorrect output. This criterion type is required for any task where sort direction is part of the correctness specification.

4.3 Baseline

The baseline generates free-form Python directly from the task description, with no scaffolding, no node registry, and no structural constraints. Each baseline model receives the same natural language task description as the compiler's planner. Each task is run N_RUNS=3 times, and success requires all three runs to pass, matching the compiler's evaluation protocol. Baseline programs run in a separate harness with a hard 30-second timeout per task; tasks requiring long-running processes such as Flask servers are handled with process group termination to prevent indefinite hang.

We evaluate two baseline models: **GPT-4.1** and **Claude Sonnet 4.6**, both at temperature=0 to reduce sampling variance. These models represent different architectural and training approaches, providing a broader baseline comparison. GPT-4.1 was used instead of GPT-4o because preliminary testing showed materially worse SQLAlchemy-version compatibility in our environment.

Each baseline model is evaluated on all 300 tasks (50 per set, 3 runs per task). Results are reported separately for each model and set in Section 5.

4.4 Evaluation Metrics

We track metrics across two dimensions: per-task granularity and per-set aggregation. All metrics are computed from three runs (N_RUNS=3) of each task, enabling analysis of both reliability and consistency.

Per-Task Metrics

Metric	Description
plan_success	LLM returned a valid, parseable JSON plan.
validation_success	Plan passed all seven validator checks (node existence, edge validity, type compatibility, acyclicity, orphan detection, input arity, required parameters).
compile_success	Compiler emitted a valid, syntactically correct Python artifact.
run_success	Artifact executed with exit code 0 (no runtime error).
criteria_passed	Output matched all task success criteria.
first_pass_success	All five stages succeeded on all three runs without needing repair. *Primary compiler metric.*
pass_count	Number of runs (out of 3) where all five stages succeeded.
avg_duration_seconds	Mean wall-clock latency across three runs (plan through execution completion).
planner_input_tokens	Total input tokens consumed by planner across three runs.
planner_output_tokens	Total output tokens generated by planner across three runs.
planner_cost_usd	USD cost of planning (GPT-4o-mini pricing at \$0.15/1M input, \$0.60/1M output).

Baseline models (GPT-4.1 and Claude Sonnet 4.6) report equivalent metrics with their own prefixes (e.g., baseline_gpt41_first_pass_success, baseline_claude_first_pass_success). Each baseline model is evaluated independently on the same tasks and success criteria.

Per-Set Aggregation

For each set of 50 tasks, we compute:

Metric	Definition
Success Rate (%)	Fraction of 50 tasks where first_pass_success=true.
Average Latency (s)	Mean of avg_duration_seconds across 50 tasks.
Total Cost (USD)	Sum of planner_cost_usd across 50 tasks.
Average Cost per Task (USD)	Total cost / 50.
Average Input Tokens	Mean of planner_input_tokens across 50 tasks.
Average Output Tokens	Mean of planner_output_tokens across 50 tasks.

Comparison Protocol

The primary comparison is **first_pass_success** (compiler) versus **first_pass_success** (baseline) aggregated per set and across all 300 tasks.

Here, first-pass success means success without any iterative correction, repair, or prompt alteration. To test whether this property holds consistently rather than accidentally, both baseline and compiler tasks are executed three times under identical conditions, and success is recorded only when all three first-pass runs succeed.

We report:

1. **Per-set success rates** for compiler, GPT-4.1, and Claude Sonnet 4.6
2. **Overall success rate** (aggregate across all six sets)
3. **Cost per successful task** (total planner cost / number of tasks passing criteria)
4. **Latency** (mean execution time, excluding tasks that timed out)

This enables analysis of: (a) how accuracy scales with pipeline depth (Sets A-D), (b) whether specialization stress tests (Sets E-F) expose systematic weaknesses, and (c) whether the compiler's reliability gains come at the cost of increased latency or planning expense.

5 Results

5.1 Main Results

Table 1 reports first-pass success rates for the compiled system, GPT-4.1 baseline, and Claude Sonnet 4.6 baseline across all six benchmark sets (n=50 per set, n=300 total).

Table 1: First-pass success rates across all benchmark sets.

Set	Focus	PlanCompiler	GPT-4.1	Claude 4.6	Δ vs GPT-4.1
A	3–5 nodes	50/50 (100%)	38/50 (76%)	30/50 (60%)	+24 pts
B	5–8 nodes	50/50 (100%)	36/50 (72%)	23/50 (46%)	+28 pts
C	8–10 nodes	44/50 (88%)	34/50 (68%)	27/50 (54%)	+20 pts
D	10+ nodes	48/50 (96%)	38/50 (76%)	36/50 (72%)	+20 pts
E	Schema traps	44/50 (88%)	20/50 (40%)	26/50 (52%)	+48 pts
F	SQL roundtrips	42/50 (84%)	36/50 (72%)	45/50 (90%)	+12 pts
Overall	**All 300**	**282/300 (94%)**	**202/300 (67%)**	**187/300 (62%)**	**+27 pts**

PlanCompiler achieves 84–100% success across all sets, with perfect 100% on Sets A and B and 96% on Set D. The compiler's advantage over GPT-4.1 ranges from 12 points (Set F) to 48 points (Set E), with an aggregate delta of 27 percentage points (94% vs 67%).

Claude Sonnet 4.6 exhibits different failure patterns: it achieves 90% on Set F (SQL roundtrips)—the only system to exceed the compiler on a set—but falls to 46% on Set B and 52% on Set E. The compiler's overall advantage over Claude is 32 percentage points (94% vs 62%).

5.2 Token Cost Analysis

Planning cost (GPT-4o-mini for compiler planning) and execution cost (GPT-4.1 and Claude for baseline generation) are reported per task and aggregated across all 300 tasks.

5.2.1 Cost Summary

Table 2: Cost efficiency across all models.

Model	Total Cost	Successes	Cost/Success	Mean Cost/Task
PlanCompiler	\$0.3562	282 (94%)	\$0.001263	\$0.001187
GPT-4.1	\$2.1396	202 (67%)	\$0.010592	\$0.007132
Claude Sonnet	\$18.3907	187 (62%)	\$0.098346	\$0.061304

The compiler is 8.4x more cost-efficient than GPT-4.1 and 77.8x more efficient than Claude on a cost-per-successful-task basis. Claude's dramatically higher cost (\$18.39 vs \$0.36) is driven by consistently higher output token generation across all sets, even on failed tasks.

5.2.2 Per-Set Token Usage

Table 3: Per-set token metrics (mean values per task).

Set	PC Input	PC Output	GPT Input	GPT Output	Claude Input	Claude Output
A	3,190	608	301	429	331	3,325
B	3,240	991	351	584	384	4,022
C	3,318	1,482	429	894	467	4,721
D	3,360	1,777	471	1,231	513	5,790
E	3,234	967	345	898	377	3,810
F	3,285	1,143	396	739	440	2,351

Key observations:

1. PlanCompiler planning tokens scale with pipeline complexity: Input tokens increase from 3,190 (Set A, simple) to 3,360 (Set D, complex). Output tokens increase from 608 to 1,777, reflecting the planning difficulty of longer pipelines.
2. Baseline execution tokens show high variance: GPT-4.1 output (429→1,231) stays bounded. Claude output (3,325→5,790) is 5-10x higher than input, indicating verbose code generation with significant redundancy.
3. Claude generates excessive output: Even on simple tasks (Set A), Claude generates 3,325 tokens while GPT-4.1 generates 429 tokens for the same input. This verbosity persists across all sets. Its relationship to Claude's failure modes is analyzed in Section 6.

5.2.3 Cost Scaling with Pipeline Depth

Table 4: Compiler cost per task increases with pipeline depth; success rates remain high.

Depth	Task Count	PC Cost	PC Success	Cost/Success
3–5 nodes	50	\$0.04	100%	\$0.00084
5–8 nodes	50	\$0.05	100%	\$0.00108
8–10 nodes	50	\$0.07	88%	\$0.00157
10+ nodes	50	\$0.08	96%	\$0.00164
Mixed (E, F)	100	\$0.11	88%	\$0.00132

PlanCompiler incurs higher planning cost on deeper pipelines while maintaining high success rates. Deeper pipelines cost more to plan while maintaining high overall accuracy. Baselines do not follow this pattern—they pay similar execution costs (\$0.0034-0.0107/task) but with highly variable success rates.

Summary

The compiler's planning investment (\$0.00119/task on average) is recovered through superior reliability. Cost per successful task is 8.4x lower than GPT-4.1 and 77.8x lower than Claude. Baseline models, despite appearing cheaper on a per-task basis, fail frequently, making their true cost-per-reliable-result substantially higher. The compiler trades higher planning cost for lower failure cost, yielding net savings in benchmark settings where first-pass correctness is important.

5.3 Latency Analysis

Wall-clock latency measures end-to-end time from task submission to success/failure determination, including plan generation, compilation, and execution.

Table 5: Mean latency per task (seconds).

Set	PlanCompiler	GPT-4.1	Claude
A	5.79s	3.57s	14.63s
B	8.54s	4.30s	16.67s
C	11.72s	5.81s	17.26s
D	12.05s	5.88s	20.75s
E	8.50s	5.64s	16.47s
F	9.64s	4.91s	11.32s
Mean	**9.37s**	**4.98s**	**16.10s**

Key findings:

1. PlanCompiler latency increases monotonically with pipeline complexity (5.79s → 12.05s, Sets A→D). This is expected: longer pipelines require longer planning and more complex code generation.

2. Baseline latency is lower on average but decoupled from success: GPT-4.1 averages 4.98s despite only 67% success rate. Claude averages 16.10s despite 62% success.

5.4 Baseline Non-Determinism

The GPT-4.1 baseline at temperature=0 showed measurable multi-run variance on repeated executions of the same tasks. On Set F, the first run achieved 36/50 success while the second run achieved 35/50—a 2% variance from identical prompts and temperature. The compiled system produced identical `app.py` output (byte-for-byte identical) across all runs of all tasks.

This provides a small but concrete example of the run-to-run variance that the architecture is designed to reduce. Even temperature=0 baseline generation exhibits multi-run variance, while deterministic compilation is reproducible by construction.

5.5 Constraint Evasion

The compiled system's failures are not randomly distributed. Across the benchmark sets where the system falls below perfect accuracy—especially C, E, and F—a systematic failure pattern emerges.

QueryEngine is the single node in the registry that accepts an unconstrained string parameter: the raw SQL query. All other node parameters are either structurally typed or validated for presence. In tasks requiring grouped aggregation, the planner systematically attempts to satisfy grouped aggregation tasks by embedding GROUP BY and COUNT(*) in the QueryEngine SQL string rather than routing the computation through the Aggregator node.

For example:

Expected: Aggregator node with `agg_func: ''count''` producing column `count`

Generated: QueryEngine with SQL `SELECT group_col, COUNT(*) cnt FROM table GROUP BY group_col` producing column `cnt`

The pipeline executes without error (the SQL is valid), but output validation fails because `file_has_column: count` criteria expect the specific column name `count`, not `cnt` or `COUNT(*)`.

Table 6: Constraint evasion accounts for 14 of 18 total compiler failures (78%).

Set	Total Failures	QueryEngine Evasion	Other Failures	Evasion %
A	0	0	0	—
B	0	0	0	—
C	6	5	1	83%
D	2	1	1	50%
E	6	4	2	67%
F	8	6	2	75%
Total	**18**	**14**	**4**	**78%**

Interpretation: In our reading, this is better explained as a consequence of partial constraint enforcement than as a generic hallucination failure. It is a predictable consequence of imposing constraints on some surfaces (node selection, parameter structure, type compatibility) while leaving one surface unconstrained (raw SQL strings). The planner routes computation through the one remaining open surface. The evasion rate is highest in Sets C, E, and F (67–83%), consistent with the hypothesis that tasks with more demanding aggregation structure make this routing behavior more likely.

6 Failure Analysis

6.1 Overview

Across 300 benchmark tasks, PlanCompiler produced 18 failures, GPT-4.1 produced 98 failures, and Claude Sonnet 4.6 produced 113 failures. PlanCompiler failures are concentrated in two identifiable patterns, both traceable to specific architectural properties. Baseline failures are broadly distributed across structural error types that compound with pipeline length and task specification clarity.

6.2 PlanCompiler Failure Modes

Pattern 1: QueryEngine Evasion (14 failures, 78% of compiler failures)

The planner satisfies aggregation tasks by embedding GROUP BY and COUNT() in the raw SQL string passed to QueryEngine, rather than constructing a pipeline that routes through the Aggregator node. The resulting output column is named COUNT(), cnt, or occurrences rather than the expected count, failing `file_has_column` criteria. The compiled artifact executes without error—exit code 0, no exceptions—making this failure invisible to `run_success` and only detectable at the criteria evaluation stage.

The evasion pattern is consistent across all sets where it appears. The planner is not hallucinating or producing structurally invalid plans. It is producing valid, executable pipelines that solve the task through the only unconstrained surface available. This failure primarily reflects an architectural property rather than a validator failure: the planner is operating within an architecture that leaves one expressive surface unconstrained. A more fully constrained IR would likely reduce or eliminate this specific failure mode.

Evasion instances are distributed unevenly across sets, suggesting that simpler tasks can be solved correctly through the constrained path, while complex aggregation tasks create sufficient pressure that the planner routes around it.

Pattern 2: Planner Graph Specification Errors (4 failures, 22% of compiler failures)

The remaining four failures are genuine planner errors—cases where the plan is structurally valid and passes all seven validator checks, but the graph specification is semantically incorrect. These failures fall into two subtypes.

The first subtype is incorrect node selection for schema drift tasks: the planner selects a structurally valid sequence of nodes that does not correctly handle column name perturbations in the input fixture. The compiled artifact runs and produces output, but the output does not satisfy the row or column criteria because the selected transformation chain is wrong for the perturbed schema.

The second subtype is incorrect parameter specification for complex multi-join aggregations: the planner provides valid parameters for each node individually but selects a join key or aggregation configuration that produces a different row count than the task requires. Both subtypes represent the boundary of what static validation can catch—the validator enforces structural correctness, not semantic correctness.

6.3 Baseline Failure Modes

6.3.1 GPT-4.1 Failures (98 failures, 32.7% of tasks)

GPT-4.1 failures are distributed across five recurring error types. The dominant failure mode is aggregation column naming (46% of failures), followed by schema drift and hallucinated columns (22%).

Failure Type	Description
Aggregation column naming	~45% Count column named occurrences, frequency, total, or num_rows instead of count. Fails file_has_column criteria.
Schema drift / hallucinated columns	~20% GPT-4.1 generates rigid schema expectations (e.g., Expected column: OrderID) not present in the fixture. Downstream validation failures.
Deprecated API patterns	~15% SQLAlchemy 1.4 engine.execute() pattern generated for SQLAlchemy 2.0 environment. Requires compatibility layer; residual failures in deeply nested calls.
Import errors / hallucinated libraries	~12% Imports of non-existent modules or incorrect function signatures from real modules. Fails at run_success with ModuleNotFoundError or AttributeError.
Structural drift across pipeline length	~8% Variable naming inconsistency between pipeline steps—a DataFrame produced in step 2 referenced under a different name in step 4. Fails silently or raises NameError.

The dominant failure mode—aggregation column naming—is entirely absent from compiler results. This is the clearest concrete demonstration of what the constraint architecture provides: when the node implementation is fixed, the model cannot produce the wrong column name regardless of how it describes the aggregation operation.

Schema drift and hallucinated column expectations illustrate a related failure mode. GPT-4.1 generates code that asserts the presence of specific column names not guaranteed by the fixture,

then fails when those columns are absent or named differently. The compiler avoids this because node templates validate structural consistency without introducing new schema constraints—the template accepts whatever columns are present and operates on them.

6.3.2 Claude Sonnet 4.6 Failures (113 failures, 37.7% of tasks)

Claude's failures are often associated with verbose code generation and additional assumptions not clearly required by the task description. Unlike GPT-4.1's systematic column-naming issues, Claude's failures stem from verbose code generation that adds features the task does not require.

Token-Based Over-Engineering Pattern:

Across all sets, failed Claude runs consume significantly more tokens than passing runs:

Set	Pass Avg Tokens	Fail Avg Tokens	Ratio
A	1,084	1,420	1.31x
B	1,271	1,637	1.29x
C	1,382	2,137	1.55x
D	2,026	2,295	1.13x
E	906	1,927	2.13x
F	797	2,131	2.67x

Failed runs are 1.13–2.67x longer than passing runs. This counter-intuitive pattern (failures associated with longer code) supports Claude's over-engineering hypothesis: when given sparse task descriptions, Claude generates unnecessary complexity—redundant error handling, custom validation logic, defensive programming—that fails because it misses the actual requirements.

Specific Failure Patterns:

1. **JSON Input Handling:** Claude generates code that outputs raw Python objects instead of proper JSON, causing `[ERROR] Top-level JSON must be an object.` errors. This occurs in 8–16% of JSON input tasks. CSV-based tasks show lower failure rates, suggesting Claude's mental model of CSV operations is more aligned with actual task intent than its JSON handling.

2. **Schema Validation Tasks:** Tasks explicitly requesting schema validation trigger a consistent failure mode: Claude generates validators that raise exceptions on the actual fixture data, treating validation as "raise if data is wrong" rather than "validate and allow." Schema validation task density increases with pipeline complexity (6% in Set A, 92% in Set E), correlating with failure rates.

3. **Logger Node Placement:** Tasks with Logger nodes occasionally trigger stderr/stdout mixing on Windows, causing exit code 1. Environment-specific but measurable (2–4 failures per set involving Logger).

6.4 Failure Mode Comparison

The structural difference between PlanCompiler and baseline failure distributions reflects the architectural difference between the two systems.

Baseline failures are broad because free-form generation has many surfaces on which errors can occur and no mechanism to catch them before execution. GPT-4.1 and Claude generate different wrong outputs for the same task (different column names, different assumptions about data format), but both are systematic in their wrongness.

PlanCompiler failures are narrow because the constraint architecture eliminates most error surfaces by construction, concentrating remaining failures at the one surface—raw SQL—that the architecture does not govern. This concentration is itself informative: 78% of compiler failures trace to the single unconstrained surface (QueryEngine SQL strings). A system with fewer unconstrained surfaces would be expected to eliminate more of these structural failure modes, though semantic planning errors could still remain. The fact that constraint evasion accounts for 78% of compiler failures suggests that closing the QueryEngine surface—by restricting its SQL grammar or adding a validator check for GROUP BY usage without a preceding Aggregator—would reduce compiler failures substantially.

In contrast, baseline failures are spread across five qualitatively different error types (column naming, schema drift, imports, structural inconsistency), indicating no clear path to improvement through single-surface fixes. Improving baseline reliability would require addressing all error surfaces simultaneously.

6.5 What Validation Catches and Does Not Catch

Failure Mode	**Caught By**	**Stage**
Hallucinated node name	CHECK 1 (node existence)	Pre-compilation
Invalid edge reference	CHECK 2 (edge validity)	Pre-compilation
Type mismatch between nodes	CHECK 3 (type compatibility)	Pre-compilation
Cyclic dependency	CHECK 4 (acyclicity)	Pre-compilation
Disconnected node	CHECK 5 (no orphans)	Pre-compilation
Fan-in wiring	CHECK 6 (input arity)	Pre-compilation
Missing required parameter	CHECK 7 (required parameters)	Pre-compilation
Invalid parameter value	Not caught	Runtime
Semantically incorrect graph	Not caught	Criteria evaluation
Unconstrained SQL string content	Not caught	Criteria evaluation

The validator provides a hard pre-compilation guarantee for structural correctness. It provides no guarantee for semantic correctness—a plan can be structurally valid and semantically wrong. This boundary is the precise limit of what static analysis can provide without executing the plan, and it is the boundary at which the remaining compiler failures occur.

6.6 Summary: Why Failures Differ

PlanCompiler failures are:

- Concentrated (78% to single surface)
- Deterministic (same task, same failure)
- Architectural (not a prompt or capability issue)
- Addressable (closing QueryEngine surface)

Baseline failures are:

- Distributed (across 5+ error types)
- Variable (same task, different failure each run)
- Less directly addressable through architecture-only fixes
- Compound (mix of column naming, schema drift, imports)

This difference demonstrates the value of constraint enforcement: errors that are diverse and unpredictable in free-form generation become concentrated and addressable in constrained systems.

7 Limitations

7.1 Dependence on LLM Planning

The system continues to rely on a large language model for initial workflow planning. While deterministic compilation improves execution reliability, incorrect plans produced by the planner can still lead to invalid workflows or execution failures. This is most visible in schema drift tasks, where incorrect graph specifications propagate through compilation.

Future work will explore planner validation and iterative repair loops to improve robustness.

7.2 Absence of Automatic Repair

The current validation layer performs static checks on graph structure, node compatibility, and parameter requirements. When validation fails, the system surfaces the error but does not yet attempt automatic plan repair.

As a result, invalid graphs terminate rather than being iteratively corrected. Incorporating a repair loop that feeds validation errors back to the planner is a natural next step.

7.3 Dependence on Node Library Coverage

System capabilities depend on the coverage and quality of the node registry. If required functionality is not represented as an available node, the planner cannot construct a valid workflow.

The current node registry is designed primarily for data processing and data pipeline tasks. While the architecture is intended to be portable across domains, the current benchmark and node implementations focus on a single domain.

Extending the registry to additional domains (e.g., APIs, document processing, or web automation) would better evaluate generality.

7.4 Prompt Token Overhead

The current implementation transmits all node descriptions to the planner, which increases prompt length and token usage as the registry grows.

Future work will explore domain-aware node selection, where the system first identifies the relevant domain and then provides only a subset of compatible nodes to the planner. This would significantly reduce token usage and improve scalability.

7.5 Limited Support for Non-Linear Workflows

The current type system and execution model primarily support linear workflow structures, where nodes execute in a single directed sequence of transformations. While the graph representation allows directed edges, the current implementation does not yet fully support more complex control-flow patterns such as branching, conditional execution, or parallel subgraphs.

As a result, the system is best suited to deterministic data-processing pipelines rather than more dynamic workflow structures. Extending the type system and execution engine to support branching graphs and richer control-flow semantics is an important direction for future work.

8 Conclusion

We have presented PlanCompiler, a deterministic compilation architecture for structured multi-step LLM pipelines that separates planning from execution through a typed node registry, a static validator, and a deterministic compiler. By restricting the language model to selecting and parameterizing nodes from a pre-verified registry, the system enforces structural correctness before any code is emitted and substantially narrows the space of possible failures.

8.1 Main Results

Across a 300-task benchmark spanning shallow and deep pipelines, SQL roundtrips, and schema-drift stress tests, PlanCompiler achieves substantially higher first-pass reliability than direct prompting baselines. This advantage persists as pipeline complexity increases and is especially strong on structurally demanding tasks, supporting the paper's central claim: deterministic

compilation prevents structural error accumulation by construction.

8.2 Failure Analysis and Architectural Boundary

The most informative remaining failures are not arbitrary. They concentrate at a single planner–semantics boundary: QueryEngine accepts unconstrained raw SQL, and the planner preferentially routes aggregation through this open surface rather than through the constrained Aggregator node. We characterize this as *constraint evasion under partial enforcement*: when most execution surfaces are constrained, planner behavior concentrates on the one that remains semantically open. This finding matters beyond the present implementation because it suggests a broader limitation of partially constrained LLM systems.

The evaluation also makes the validator boundary explicit. Structural failures are eliminated pre-compilation, while semantically incorrect but structurally valid plans remain possible. The residual failures therefore do not undermine the architecture's guarantees; they identify their exact limit.

Baseline models fail differently. GPT-4.1 is more prone to schema and naming errors, while Claude Sonnet 4.6 performs relatively well on SQL-heavy tasks but remains less reliable overall. These distinct failure signatures suggest that baseline reliability is not a single problem with a single remedy. PlanCompiler's contribution is not only to improve success rates, but to shift failures into a smaller, more interpretable class.

Despite added planning structure, the compiler also yields a substantially lower cost per successful task than either baseline. For structured workflow generation, deterministic compilation offers a practical path toward making LLM systems more reliable, more inspectable, and more predictable.

8.3 Future Work

Five directions follow naturally from this work.

First, closing the QueryEngine surface—for example by restricting its accepted SQL grammar or adding a validator check that flags `GROUP BY` usage without a preceding `Aggregator`—would likely yield the largest immediate reliability gain.

Second, extending the benchmark to additional domains would test whether the architecture transfers beyond the current data-pipeline setting. The present evaluation focuses on structured data workflows such as CSV/JSON ingestion, transformation, aggregation, and SQL persistence.

Third, planner-model ablation would test multiple language models as the planning layer to identify both the cheapest reliable planner and the most accurate one. The current implementation uses GPT-4o-mini for cost efficiency; broader comparison would make the accuracy–cost tradeoff explicit.

Fourth, adding a structured repair loop would allow the planner to revise invalid plans in response to validator feedback under bounded retry. This would address a practical limitation of

the current one-shot design, although it would not remove the underlying semantic boundary identified in the failure analysis.

Fifth, comparative evaluation against other structured code-generation systems, such as LangGraph, LLMCompiler, and text-to-pipeline approaches, would help locate PlanCompiler more precisely within the broader design space of constrained LLM program synthesis.

8.4 Broader Implications

Reliability in multi-step LLM workflows is often treated as a prompting problem—a matter of better instructions, chain-of-thought scaffolding, or output parsing. The results here suggest that, for this class of structured multi-step tasks, reliability can be productively treated as a compiler problem: identify the structural failure modes, close them by construction, and report precisely what the boundary does and does not guarantee.

The residual failures then become informative rather than opaque.

The distinction between structural and semantic correctness is the key insight. Static analysis catches structural errors by construction. Semantic errors—incorrect graph specifications, wrong parameter choices—require either human review, test-based validation, or repair loops. The architecture presented here makes this distinction explicit and measurable, enabling precise conversation about what has been gained and what remains open.

Acknowledgments

We thank the reviewers for their detailed feedback and suggestions for improvement.

References

[1] LangChain, Inc. Langchain, 2026. URL `https://www.langchain.com/`. Official project website, accessed 2026-03-19.

[2] LlamaIndex. Llamaindex, 2026. URL `https://developers.llamaindex.ai/python/framework/`. Official documentation, accessed 2026-03-19.

[3] Hugging Face. smolagents documentation, 2026. URL `https://huggingface.co/docs/smolagents/index`. Official documentation, accessed 2026-03-19.

[4] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling. *arXiv preprint arXiv:2312.04511*, 2023. doi: 10.48550/arXiv.2312.04511. URL `https://arxiv.org/abs/2312.04511`.

[5] Yifu Lu, Shengjie Liu, and Li Dong. Orchdag: Complex tool orchestration in multi-turn interactions with plan dags. *arXiv preprint arXiv:2510.24663*, 2025. URL `https://arxiv.org/abs/2510.24663`.

[6] Yuhang Ge, Yachuan Liu, Zhangyan Ye, Yuren Mao, and Yunjun Gao. Text-to-pipeline: Bridging natural language and data preparation pipelines. *arXiv preprint arXiv:2505.15874*, 2025. doi: 10.48550/arXiv.2505.15874. URL `https://arxiv.org/abs/2505.15874`.

[7] Rohan Bavishi, Caroline Lemieux, Roy Fox, Koushik Sen, and Ion Stoica. Autopandas: Neural-backed generators for program synthesis. *Proceedings of the ACM on Programming Languages*, 3

(OOPSLA):168:1–168:27, 2019. doi: 10.1145/3360594. URL https://dl.acm.org/doi/10.1145/3360594.

[8] Wes McKinney. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.

[9] Naman Jain, Skanda Vaidyanath, Arun Iyer, Nagarajan Natarajan, Suresh Parthasarathy, Sriram K. Rajamani, and Rahul Sharma. Jigsaw: Large language models meet program synthesis. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, 2022. URL https://arxiv.org/abs/2112.02969.

[10] Zahra Moslemi, Keerthi Koneru, Yen-Ting Lee, Sheethal Kumar, and Ramesh Radhakrishnan. Polaris: Typed planning and governed execution for agentic ai in back-office automation. *arXiv preprint arXiv:2601.11816*, 2026. doi: 10.48550/arXiv.2601.11816. URL https://arxiv.org/abs/2601.11816.

A Full Node Registry

Table 7 provides the complete node registry with all 26 nodes across six categories.

Table 7: Complete node registry, 26 nodes across six categories.

Node	**Category**	**Input**	**Output**	**Required Params**
CSVParser	Ingestion	FilePath	DataFrame	file_path
JSONParser	Ingestion	FilePath	DataFrame	file_path
ExcelParser	Ingestion	FilePath	DataFrame	file_path
SchemaValidator	Processing	DataFrame	DataFrame	—
DataTransformer	Processing	DataFrame	DataFrame	—
DataFilter	Processing	DataFrame	DataFrame	condition
ColumnSelector	Processing	DataFrame	DataFrame	columns
NullHandler	Processing	DataFrame	DataFrame	strategy
DataSorter	Processing	DataFrame	DataFrame	by, ascending
TypeCaster	Processing	DataFrame	DataFrame	mapping
DataFrameJoin	Processing	DataFrame	DataFrame	on, how
StatsSummary	Processing	DataFrame	DataFrame	—
DataDeduplicator	Processing	DataFrame	DataFrame	—
Aggregator	Processing	DataFrame	DataFrame	group_by, agg_func
SQLiteConnector	Storage	DataFrame	DBHandle	db_path, table_name
SQLiteReader	Storage	FilePath	DBHandle	db_path
PostgresConnector	Storage	DataFrame	DBHandle	connection_string, table_name
QueryEngine	Storage	DBHandle	DataFrame	query
CSVExporter	Export	DataFrame	FilePath	output_path
JSONExporter	Export	DataFrame	FilePath	output_path
RESTEndpoint	API/HTTP	DBHandle	HTTPResponse	route, port
AuthMiddleware	API/HTTP	HTTPResponse	HTTPResponse	api_key_env_var
ErrorHandler	API/HTTP	ANY	HTTPResponse	—
Logger	Observability	ANY	ANY	—
HTTPToDataFrame	Adapter	HTTPResponse	DataFrame	—

Design notes.

DataSorter requires both by and ascending as required parameters with no defaults. This was changed in v4.0 after tasks specifying descending sort silently passed `file_has_column` criteria while producing incorrectly ordered output. Making ascending required forces the validator to reject any plan that omits it.

SQLiteReader is an entry-point node only. Its input_type is FilePath as a registry convention—it opens a pre-existing .db file from disk via the db_path parameter and must only appear at the start of a pipeline. The correct write-then-query pattern is SQLiteConnector → QueryEngine. Placing SQLiteReader after SQLiteConnector produces a TYPE_MISMATCH (DBHandle ≠ FilePath) which the validator correctly catches.

Aggregator always produces a column named count for count aggregations. This is a fixed property of the node implementation, not a parameter. Tasks requiring a count column must use the Aggregator node; pipelines that push GROUP BY COUNT(*) into the QueryEngine SQL string produce a column named COUNT(*) which fails `file_has_column: count` criteria.

Logger accepts and emits ANY, allowing it to be inserted at any point in a pipeline as a transparent observability probe without breaking type compatibility. Prior to v5.0, CHECK 3 only exempted edges where the target's input type was ANY, incorrectly raising TYPE_MISMATCH on Logger outputs wired to typed nodes. The v5.0 fix extended the exemption to both directions.

Planner also contains an additional field: glue_code (an optional override for the auto-generated execution block; plans using non-trivial glue code are tracked separately and excluded from the reported reliability results).

B Example Compiled Artifact

The following is a representative compiled `app.py` output for the task "Read a CSV file, filter rows where salary exceeds 40000, store in SQLite, query the result, and export to CSV." The artifact is self-contained—it has no imports from the compiler or registry at runtime.

Listing 1: Example compiled PlanCompiler artifact

```python
# Generated by PlanCompiler
# DO NOT EDIT MANUALLY
# --- Node: CSVParser ---
import pandas as pd
def csv_parser(file_path: str) -> pd.DataFrame:
    df = pd.read_csv(file_path)
    print(f"[CSVParser] Loaded {len(df)} rows from {file_path}")
    return df

# --- Node: DataFilter ---
def data_filter(df: pd.DataFrame, condition: str) -> pd.DataFrame:
    result = df.query(condition)
    print(f"[DataFilter] {len(result)} rows after filter: {condition}")
```

```python
    return result

# --- Node: SQLiteConnector ---
import sqlite3
def sqlite_connector(df: pd.DataFrame, db_path: str,
                     table_name: str) -> sqlite3.Connection:
    conn = sqlite3.connect(db_path)
    df.to_sql(table_name, conn, if_exists='replace', index=False)
    print(f"[SQLiteConnector] Written {len(df)} rows to {table_name}")
    return conn

# --- Node: QueryEngine ---
def query_engine(conn: sqlite3.Connection, query: str) -> pd.DataFrame:
    result = pd.read_sql_query(query, conn)
    print(f"[QueryEngine] Returned {len(result)} rows")
    return result

# --- Node: CSVExporter ---
def csv_exporter(df: pd.DataFrame, output_path: str) -> str:
    df.to_csv(output_path, index=False)
    print(f"[CSVExporter] Written to {output_path}")
    return output_path

# --- Execution (auto-generated) ---
if __name__ == '__main__':
    out_csv_parser = csv_parser(file_path='employees.csv')
    out_data_filter = data_filter(out_csv_parser,
                                  condition='salary > 40000')
    out_sqlite_connector = sqlite_connector(out_data_filter,
                                            db_path='out.db',
                                            table_name='employees')
    out_query_engine = query_engine(out_sqlite_connector,
                                    query='SELECT * FROM employees')
    out_csv_exporter = csv_exporter(out_query_engine,
                                    output_path='output.csv')
    print(out_csv_exporter)
```

This artifact illustrates the key properties of the execution model: inter-node execution state passes through function return values, there is no shared mutable in-memory state, no global context, and no implicit passing, and the execution block is entirely auto-generated from the validated plan with no LLM involvement.

C Benchmark Fixture Reference

All benchmark tasks draw from five fixed fixture files. Row counts are documented precisely because `file_row_count` criteria depend on them.

Fixture	Total Rows	Key Filter Counts
sales.csv	40 (38 after dedup)	revenue > 100: 27 rows
products.json	25	price > 50: 15 rows
employees.csv	23	salary > 40,000: varies by task
predictions.csv	30	score > 0.5: varies by task
events.json	50	—

Fixture files are fixed across all benchmark runs. Tasks are parameterized against these fixtures at authoring time; fixture row counts were verified against actual file contents.

D Reproduction

The complete system and benchmark are available at github.com/prnvh/plancompiler. To reproduce results:

Listing 2: Reproduction steps

```
git clone https://github.com/prnvh/plancompiler
cd plancompiler
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

cat > .env <<EOF
OPENAI_API_KEY=your_key_here
ANTHROPIC_API_KEY=your_key_here
EOF


\# Run compiler benchmark (per set)
python benchmark/harness.py --tasks benchmark/tasks/tasks\_set\_a.json
    --output benchmark/results/results\_set\_a.json
python benchmark/harness.py --tasks benchmark/tasks/tasks\_set\_b.json
    --output benchmark/results/results\_set\_b.json
python benchmark/harness.py --tasks benchmark/tasks/tasks\_set\_c.json
    --output benchmark/results/results\_set\_c.json
python benchmark/harness.py --tasks benchmark/tasks/tasks\_set\_d.json
    --output benchmark/results/results\_set\_d.json
python benchmark/harness.py --tasks benchmark/tasks/tasks\_set\_e.json
    --output benchmark/results/results\_set\_e.json
python benchmark/harness.py --tasks benchmark/tasks/tasks\_set\_f.json
    --output benchmark/results/results\_set\_f.json

\# Run baseline (per set, after compiler)

python benchmark/run\_baseline.py --tasks benchmark/tasks/tasks\_set\_a
    .json --results benchmark/results/results\_set\_a.json
```

```
python benchmark/run\_baseline.py --tasks benchmark/tasks/tasks\_set\_b
    .json --results benchmark/results/results\_set\_b.json
python benchmark/run\_baseline.py --tasks benchmark/tasks/tasks\_set\_c
    .json --results benchmark/results/results\_set\_c.json
python benchmark/run\_baseline.py --tasks benchmark/tasks/tasks\_set\_d
    .json --results benchmark/results/results\_set\_d.json
python benchmark/run\_baseline.py --tasks benchmark/tasks/tasks\_set\_e
    .json --results benchmark/results/results\_set\_e.json
python benchmark/run\_baseline.py --tasks benchmark/tasks/tasks\_set\_f
    .json --results benchmark/results/results\_set\_f.json
```